Mathematical Manual & Tutorial — Densidade Vertical

This document is the **canonical reference** for **equations, algorithms, and models** implemented in the codebase, plus a **pedagogical tutorial** for reading results and tuning parameters. It complements the narrative Technical Manual (architecture and examples).

LaTeX format (StackEdit, MathJax, KaTeX, GitHub)

All mathematics is written in **LaTeX** using:

Delimiter	Use
`$ ... $`	Inline math, e.g. `$f(m) = 440 \cdot 2^{(m-69)/12}$`
`$$` on its own line	Display (block) math: open with `$$`, equation, close with `$$` on a new line

This matches **StackEdit**, **Stack Exchange** (MathJax), **VS Code** (Markdown Math), **GitHub**, and **KaTeX**. Do not use bare Unicode superscripts or the `·` character for multiplication inside formulas; use `\cdot`, `\times`, or `\log_{10}(1+x)` as below.

Table of contents

---

1. Notation and conventions

Symbol	Meaning
m_i	MIDI pitch (real-valued; microtones allowed)
f	Frequency in Hz
n	Number of notes in the vertical (same as `len(notes)`)
a_i	Non-negative weight for pitch i (instrument densities or combination-tone weights)
$S = \sum_i a_i$	Total weight (for spectral moments)
λ	Decay parameter for interval density (`DEFAULT_LAMBDA` or calibrated)
δ	Distance in **microtonal steps** (24 per octave in this system); linked to semitone distance Δ_{st} by $\delta = 2\Delta_{\text{st}}$ in the default path

The application is **symbolic**: it does not analyse audio waveforms; it computes metrics from **note names**, **dynamics**, and **instrument tags**.

---

2. Models by item (formula catalog)

A. Pitch and frequency

Module: `microtonal.py` (`midi_to_hz`, `hz_to_midi`, `note_to_midi`, …).

Equal temperament (continuous MIDI):

$$f(m) = f_{\text{A4}} \cdot 2^{(m - m_{\text{A4}})/12}, \quad f_{\text{A4}} = 440\,\text{Hz},\ m_{\text{A4}} = 69.$$

Inverse:

$$m = m_{\mathrm{A4}} + 12 \log_2\left(\frac{f}{f_{\mathrm{A4}}}\right), \quad f > 0.$$

Semitone span of a chord:

$$A_{\mathrm{st}} = \max_i m_i - \min_i m_i \quad \text{(updated when combination tones widen the span)}.$$

B. Interval density (pairwise decay)

Module: `densidade_intervalar.py` — `modified_exponential_decay`, `calculate_interval_density`.

Microtonal distance for a pair (i, j):

$$\Delta_{\mathrm{st}}(i,j) = |m_i - m_j|, \qquad \delta(i,j) = 2 \cdot \Delta_{\mathrm{st}}(i,j).$$

(If two *different* note strings yield $\Delta_{\mathrm{st}} < 0.01$ semitones due to float noise, the code may force a minimum step of 0.25 semitones before computing δ.)

Decay (unison = strongest contribution):

$$\phi(\delta;\lambda) = \begin{cases} 1 & \delta = 0 \\ e^{-\lambda\delta} & \delta > 0 \end{cases}$$

with λ from `load_calibrated_parameters()` or an explicit argument.

Raw interval density (sum over unordered pairs):

$$D_{\mathrm{int}}^{\mathrm{raw}} = \sum_{i<j} \phi\big(\delta(i,j);\lambda\big) \cdot (\text{optional perceptual weight}).$$

Psychoacoustic wrapper (`calculate_interval_density_psychoacoustic`) first uses the **scalar** returned by `calculate_interval_density` (the total sum above). Let $T = D_{\mathrm{int}}^{\mathrm{raw}}$. For $n \geq 2$:

$$\bar{D}_{\mathrm{int}} = \frac{2T}{n(n-1)} \quad \text{(average contribution per unordered pair)}.$$

If `USE_LOG_COMPRESSION` is true (`config.py`):

$$\bar{D}_{\mathrm{int}} \leftarrow \log_{10}(1 + \bar{D}_{\mathrm{int}}).$$

C. Optional perceptual weighting (pairwise)

Module: `densidade_intervalar.py — calculate_microtonal_perceptual_weight(midi1, midi2, delta_semitons)`.

This is a **heuristic multiplier** $\pi(\text{register}, \Delta_{\text{st}})$ applied to each pair's ϕ before summing.

Let $m_{mid} = (m_i + m_j)/2$ (register proxy) and $\Delta = \Delta_{\text{st}}(i, j)$.

Start with $\pi = 1$.

1. **Register** (piecewise):
   - if $m_{mid} > 84$: $\pi \leftarrow 1.3\pi$
   - elif $m_{mid} > 72$: $\pi \leftarrow 1.1\pi$
   - elif $m_{mid} < 48$: $\pi \leftarrow 0.8\pi$
2. **Interval size** (piecewise on Δ):
   - if $\Delta \leq 1$: $\pi \leftarrow 1.5\pi$
   - elif $\Delta \leq 2$: $\pi \leftarrow 1.3\pi$
   - elif $\Delta \leq 4$: $\pi \leftarrow 1.1\pi$
   - elif $\Delta \geq 12$: $\pi \leftarrow 0.9\pi$

Pair contribution: $\phi(\delta; \lambda) \cdot \pi$.

D. Interval density — psychoacoustic path

Module: `densidade_intervalar.py — calculate_interval_density_psychoacoustic` (after $\bar{D}_{\text{int}}$ is formed).

If `use_psychoacoustic` is false, return $\bar{D}_{\text{int}}$.

Otherwise, with unit amplitudes $a_i = 1$:

1. **Masking:** `amps_masked = critical_band_masking(pitches, amps)`
2. **Roughness:** `rough_vec = calculate_roughness(pitches, amps_masked)` (vector; scalar `rough_gain` is derived in `data_processor` path — see module `densidade_intervalar` for the exact scalar used in the interval-density branch)
3. **Loudness:** `loud_gain = mean(apply_loudness_correction(pitches, amps_masked))`

In `calculate_interval_density_psychoacoustic` the implementation uses:

```
masking_gain = mean(amps_masked)
rough_gain   = 1 + sum(rough_vec) / sqrt(n)
dens_final   = dens_scalar * masking_gain * rough_gain * loud_gain
```

and optionally another $\log_{10}(1+x)$ compression if `USE_LOG_COMPRESSION` is true (x is the scalar density before that step).

E. Psychoacoustic primitives (Bark, masking, roughness, loudness)

Module: `psychoacoustic_corrections.py`.

E.1 Bark scale (Zwicker & Terhardt–type)

For $f > 0$:

$$z(f) = 13 \arctan(0.00076 f) + 3.5 \arctan\left(\left(\frac{f}{7500}\right)^2\right).$$

Constants: `BARK_SCALE_FACTOR = 13`, `BARK_SCALE_FREQ1 = 0.00076`, `BARK_SCALE_FREQ2 = 7500`.

E.2 Critical-band masking (simplified)

For each pair (i, j), iej, if $|z(f_i) - z(f_j)| < 1$ Bark and $a_j > a_i$, then amplitude i is attenuated:

$$a_i \leftarrow a_i \cdot \big(1 - (1 - |z_i - z_j|) \cdot s_{\text{mask}}\big), \quad s_{\text{mask}} = 0.25 \text{ (default)}.$$

with default `masking_slope = 0.25`. Implemented as a loop over ordered pairs.

E.3 Roughness (simplified Plomp–Levelt / Sethares-style)

For each unordered pair (i, j), $f_i \le f_j$, define relative spacing in percent of the mean frequency:

$$r = \frac{|f_i - f_j|}{(f_i + f_j)/2} \times 100.$$

Only pairs with $r < 30$ contribute. For $x = r/6.5$:

$$\rho_{ij} = \begin{cases} x \cdot e^{1-x} & x < 1 \\ e^{-0.5(x-1)} & x \ge 1 \end{cases}$$

Weighted by $\min(a_i, a_j)$. The function returns **one scalar** sum over pairs (or a vector in alternate variants — see source for the exact return type used in each call site).

E.4 Equal-loudness (simplified)

`equal_loudness_correction(f)` returns a piecewise multiplier (boost below ~200 Hz, transition to 1 kHz, high-frequency boost above 4 kHz with rolloff above 10 kHz), clamped to ≥ 0.1.

`apply_loudness_correction` multiplies each amplitude by the correction at its frequency.

F. Instrument density and sonic mass

Modules: `instrumentos/*`, `data_processor.py`.

Per note (dynamic mapped to a known level if needed):

$$d'_i = d_i \cdot \sqrt{n_{\text{instr},i}}.$$

Total instrument density:

$$D_{\text{inst}} = \sum_i d'_i.$$

Sonic mass (`calcular_massa_sonora`): dynamic factors $g(\text{dynamic}) \in \{0.2, \dots, 3.0\}$ from `config.py` mapping (`pppp` … `ffff`).

$$M = \sum_i d'_i \cdot g_i \cdot n_{\text{instr},i}, \qquad \text{boost} = \sqrt{M}.$$

G. Weighted density (normalisation + Stevens)

Module: `data_processor.py` — `calcular_densidade_ponderada_normalizada`.

Min–max (default):

$$\widehat{D}_{\text{inst}} = \frac{D_{\text{inst}}}{D_{\text{inst,max}}}, \quad \widehat{D}_{\text{int}} = \frac{D_{\text{int}}}{D_{\text{int,max}}}.$$

Stevens (optional): $\widetilde{D}_{\text{inst}} = \widehat{D}_{\text{inst}}^{\alpha}$, $\widetilde{D}_{\text{int}} = \widehat{D}_{\text{int}}^{\beta}$ for positive normalised values.

Blend:

$$D_{\text{pond}} = 10 \cdot \left(w\, \widetilde{D}_{\text{inst}} + (1 - w)\, \widetilde{D}_{\text{int}} \right), \quad w \in [0, 1].$$

H. Refined density, cohesion, complexity, total density

Module: `data_processor.py` — `calculate_metrics`.

Refined:

$$D_{\text{ref}} = \begin{cases} D_{\text{pond}} / A_{\text{st}} & A_{\text{st}} > 0 \\ D_{\text{pond}} & A_{\text{st}} = 0 \end{cases}$$

Spectral entropy H from extended moments (see §I). **Complexity factor:**

$$C_{\text{comp}} = 1 + \ln(1 + H).$$

Cohesion:

$$C_{\text{coes}} = \frac{10}{1 + A_{\text{st}}}.$$

Total (before global normalisation):

$$D_{\text{total}}^{\text{raw}} = D_{\text{ref}} \cdot C_{\text{coes}} \cdot C_{\text{comp}} \cdot (1 - 0.15 \cdot \text{harmonicRatio}) \cdot \sqrt{M}.$$

Normalise: divide by `MAX_DENS_GLOBAL`. Optionally apply $\log_{10}(1 + x)$ again if `USE_LOG_COMPRESSION` (x = normalised total density).

Absolute density (reference):

$$D_{\text{abs}} = D_{\text{pond}} \cdot \ln(1 + N_{\text{tones}}).$$

I. Spectral moments, chroma, harmonic ratio

Module: `spectral_analysis.py`.

Moments on pitches m_i with weights a_i:

$$\mu = \frac{1}{S} \sum_i a_i m_i, \quad \sigma^2 = \frac{1}{S} \sum_i a_i (m_i - \mu)^2, \quad \gamma_1 = \frac{\frac{1}{S} \sum_i a_i (m_i - \mu)^3}{\sigma^3} \ (\sigma > 0).$$

Excess kurtosis:

$$\gamma_2 = \frac{\frac{1}{S} \sum_i a_i (m_i - \mu)^4}{\sigma^4} - 3.$$

Flatness: ratio of geometric to arithmetic mean of amplitudes (on $a_i > 10^{-10}$).

Roll-off (85%): cumulative sum of a_i in array order; MIDI at 85% cumulative energy mapped to Hz.

Entropy: $H = -\sum_i p_i \log_2 p_i$, $p_i = a_i / S$, with small p_i filtered.

Chroma: $c_i = \text{round}(m_i) \bmod 12$, energies summed per class and normalised.

Harmonic ratio: fundamental $m_{\min}$; energy in bins with $(m_i - m_{\min}) \bmod 12 \approx 0$ (atol `0.25`).

J. Texture, timbre blend, orchestration

Module: `timbre_texture_analysis.py`.

Texture (`calculate_texture_density`):

- `average_texture_density` $= \sum_i n_{\text{instr},i}$
- `texture_polyphony` $= \frac{1}{n}\sum_i n_{\text{instr},i}$
- `texture_variability` $= \text{std}(m_i)$
- `texture_contrast` $= \max m_i - \min m_i$

Timbre blend (`calculate_timbre_blend`):

- `timbre_diversity` $= |\{\text{unique instruments}\}|/n$
- For each instrument k, average density $\bar{d}_k$; `density_variance` $= \text{Var}_k(\bar{d}_k)$
- `blend_index` $= 1/(1+\sigma_d^2)$ where $\sigma_d^2 = \text{Var}_k(\bar{d}_k)$ (`density_variance` in code)

Orchestration (`calculate_orchestration_balance`): split registers **baixo** $[0, 48)$, **médio** $[48, 72)$, **agudo** $[72, 108)$. Sum densities per register; normalise to p_r, $r \in \{1, 2, 3\}$.

- **Register balance** (normalised entropy):

$$R_{\text{reg}} = \frac{-\sum_{r:p_r>0} p_r \log_2 p_r}{\log_2 3}.$$

- **Density balance:** `1 - (max(p) - min(p))` over the three registers.
- **Gini** on sorted normalised register masses; **evenness** $= 1 - |\text{Gini}|$.

K. Combination tones (non-linear distortion model)

Module: `combination_tones.py`.

Signal model: $y = x + a_2 x^2 + a_3 x^3$ with $x(t) = A_1 \cos \omega_1 t + A_2 \cos \omega_2 t$.

Linear amplitudes from dB: $A = 10^{(\text{dB}-70)/20}$.

Frequencies: $f_2 - f_1$, $2f_1 - f_2$, $2f_2 - f_1$, optional $f_1 + f_2$.

Weights:

- $|a_2| A_1 A_2$ for diff and sum
- $|a_3| \cdot \frac{3}{4} A_1^2 A_2$ for $2f_1 - f_2$
- $|a_3| \cdot \frac{3}{4} A_2^2 A_1$ for $2f_2 - f_1$

Filter by `[freq_min, freq_max]` and `min_weight`. Resultants are merged into the same pitch/density arrays used for spectral metrics in `calculate_metrics`.

L. λ calibration

Module: `densidade_intervalar.py` — `calibrate_lambda`.

Reference ratings `CONSONANCE_RATINGS` map interval classes to empirical scores. For candidate λ, minimise squared error between predicted normalised density and ratings (see Technical Manual, Section 3.15). Optimal λ is saved to `config/density_params.json`.

---

3. End-to-end pipeline (diagram)

```
flowchart TD
  subgraph Input
    N[Notes + dynamics + instruments + counts]
  end

  subgraph Interval
    DI[Interval density: pairwise decay]
    DIP[Optional psychoacoustic interval scaling]
  end

  subgraph Instrument
    INST[Per-note instrument densities]
    DINST[Sum + sqrt(num_instruments)]
  end

  subgraph Fusion
    W[Weighted + Stevens + min-max]
    R[Refined density / spread]
  end

  subgraph Combo
    CT[Combination tones optional]
  end

  subgraph Spectral
    SP[Spectral moments + chroma + harmonic ratio]
    TX[Texture + timbre + orchestration]
  end

  subgraph Out
```

```
  TOT[Total density + absolute + outputs]
end

N --> DI
DI --> DIP
N --> INST --> DINST
DIP --> W
DINST --> W
W --> R
N --> CT
R --> SP
CT --> SP
N --> TX
SP --> TOT
TX --> TOT
```

---

4. Pedagogical tutorial

4.1 What problem does this solve?

Vertical density tries to quantify how "full" or "complex" a **simultaneity** (chord or cluster) is **at one moment in time**, using:

- **Interval content** (how intervals are spaced — exponential decay favours small δ in ϕ),
- **Orchestration** (which instruments and how many),
- **Spectral shape** (moments, entropy, harmonic ratio),
- Optionally **psychoacoustic** corrections and **combination tones**.

It is **not** voice-leading analysis, **not** rhythmic density, and **not** a full hearing model — it is a **consistent, tunable metric** for composition and analysis.

4.2 How to read one number

1. Look at `density.interval` vs `density.instrument` : is the chord "heavy" because of **intervals** or **timbre**?
2. Check `density.weighted` and `density.refined` : refined divides by pitch span — **wide spreads** reduce refined density unless compensated elsewhere.
3. **Spectral entropy** drives `complexity` in the total-density formula — **more spread-out** spectral distributions (higher entropy) increase the factor C_{comp}.
4. **Harmonic ratio** (0–1) **reduces** total density slightly via $(1 - 0.15 \cdot \text{harmonicRatio})$ — energy in harmonic rows relative to the lowest pitch is treated as "simpler".

4.3 Lesson 1 — Minimal chord

Input three notes in the same octave, `weight_factor = 0.5`, `use_stevens = True`. Compare:

- **Major triad** vs **cluster** (minor seconds): interval density should be **higher** for the cluster (more close pairs, larger ϕ).
- Increase **instrument** weight by setting `weight_factor` toward **1** — instrument modules dominate.

4.4 Lesson 2 — Parameters that matter most

Parameter	Effect
`weight_factor` (w)	Balance instrument vs interval density
`alpha`, `beta` (Stevens)	Non-linear compression of normalised densities
`use_psychoacoustic`	Extra masking / roughness / loudness on interval path
`use_perceptual_weighting`	Heuristic register + interval-size weights on pairs
`calculate_combination_tones`	Adds virtual pitches for spectral/total density
`USE_LOG_COMPRESSION`	Flattens extreme values (config)

4.5 Lesson 3 — Calibration

If you use **λ calibration**, run the calibration workflow from the menu (Tools → Calibration) or call `calibrate_lambda` with your own experimental data. The fitted λ changes how fast ϕ decays with δ — **larger λ** penalises wide intervals more strongly.

4.6 Mini exercise (paper)

For two notes $m_1 = 60$, $m_2 = 64$, $\lambda = 0.05$: compute $\delta = 8$, $\phi(\delta) = e^{-0.05 \cdot 8}$. For three notes, sum **three** pairwise terms. Compare with $\lambda = 0.2$.

---

5. Glossary

Term	Meaning
Vertical	One time-slice of simultaneous notes (not a score)
MIDI	Pitch in semitones; fractional MIDI = microtones
Microtonal steps	Internal 24-step octave grid; $\delta = 2 \cdot \Delta_{\mathrm{st}}$ in default pairing
Stevens	Power-law on **normalised** densities to mimic perceptual compression
Combination tones	Virtual partials from nonlinear distortion — **not** recorded audio
Bark	Critical-band rate scale for masking
Roughness	Sensory dissonance proxy for close partials

---

6. Code index

Topic	Primary file(s)
Main pipeline	`data_processor.py` `calculate_metrics`
Interval decay	`densidade_intervalar.py`
Psychoacoustics	`psychoacoustic_corrections.py`
Spectral metrics	`spectral_analysis.py`
Texture / orchestration	`timbre_texture_analysis.py`
Combination tones	`combination_tones.py`
Pitch conversion	`microtonal.py`
Instruments	`instrumentos/*.py`

For architecture and output JSON keys, see TECHNICAL_MANUAL.md. For function signatures, see API.md.

Last updated to align with the codebase structure (symbolic pipeline, optional psychoacoustics and combination tones).